Aluna: Mirelle Casimiro Silvino

Disciplina: Sistemas Operacionais

Atividade 5 - Filas de mensagens POSIX

1. Quantas filas são necessárias?

São necessárias duas filas. Uma fila POSIX serve para comunicação em um sentido: se A e B usassem uma única fila para pedidos e respostas, A poderia ler de volta o seu próprio pedido (ou B poderia ler uma resposta) e o fluxo ficaria confuso. Por isso foram usadas:

- /fila-mirelle-pedido: A envia os números n1 e n2, B recebe.
- /fila-mirelle-resposta: B envia a soma, A recebe.

2. A implementação:

Os dois processos abrem as filas com mq_open e a flag O_CREAT, então a ordem de início não importa (quem chegar primeiro cria a fila). As mensagens são estruturas de tamanho fixo (pedido_t com dois inteiros; a resposta é um inteiro). Cada processo é compilado separadamente, com gcc processo_a.c -o processo_a -lrt e gcc processo_b.c -o processo_b -lrt. Para o caso de vários processos A, também foi feita a versão corrigida (processo_a_corrigido.c e processo_b_corrigido.c), descrita no item 7.

filas.h

```
#ifndef FILAS_H
#define FILAS_H
#include <fcntl.h>
#include <mqueue.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <time.h>

#define FILA_PEDIDO   "/fila-mirelle-pedido"    // A -> B
#define FILA_RESPOSTA "/fila-mirelle-resposta"  // B -> A (versao basica)

// mensagem de pedido (versao basica: apenas os dois numeros)
typedef struct { int n1, n2; } pedido_t;

// mensagem de pedido (versao corrigida: identifica quem pediu)
typedef struct { pid_t pid; int n1, n2; } pedido_id_t;

static inline mqd_t abre_fila (const char *nome, int flags, size_t tam)
{
   struct mq_attr attr;
   attr.mq_flags = 0;
   attr.mq_maxmsg = 10;
   attr.mq_msgsize = tam;
   attr.mq_curmsgs = 0;
   mqd_t q = mq_open (nome, flags | O_CREAT, 0666, &attr);
   if (q == (mqd_t) -1) { perror ("mq_open"); exit (1); }
   return q;
}
#endif
```

processo_a.c (processo A, versão básica)

```
// Processo A (versao basica: 2 filas)
#include "filas.h"

int main (void)
{
   srandom (time (NULL) ^ getpid ());
```

```
   mqd_t pedido   = abre_fila (FILA_PEDIDO,   O_WRONLY, sizeof (pedido_t));
   mqd_t resposta = abre_fila (FILA_RESPOSTA, O_RDONLY, sizeof (int));

   while (1)
   {
      pedido_t p = { random () % 100, random () % 100 };
      int soma;
      if (mq_send (pedido, (char *) &p, sizeof (p), 0) < 0) { perror ("mq_send"); exit (1); }
      if (mq_receive (resposta, (char *) &soma, sizeof (soma), NULL) < 0) { perror
("mq_receive"); exit (1); }
      printf ("A[%d]: %d + %d = %d%s\n", getpid (), p.n1, p.n2, soma,
              (p.n1 + p.n2 == soma) ? "" : "   <-- ERRADO (soma de outro pedido)");
      fflush (stdout);
   }
}
```

processo_b.c (processo B, versão básica)

```
// Processo B (versao basica: 2 filas)
#include "filas.h"

int main (void)
{
   mqd_t pedido   = abre_fila (FILA_PEDIDO,   O_RDONLY, sizeof (pedido_t));
   mqd_t resposta = abre_fila (FILA_RESPOSTA, O_WRONLY, sizeof (int));

   while (1)
   {
      pedido_t p;
      if (mq_receive (pedido, (char *) &p, sizeof (p), NULL) < 0) { perror ("mq_receive");
exit (1); }
      int soma = p.n1 + p.n2;
      printf ("B[%d]: recebeu %d e %d\n", getpid (), p.n1, p.n2);
      fflush (stdout);
      sleep (1);
      if (mq_send (resposta, (char *) &soma, sizeof (soma), 0) < 0) { perror ("mq_send"); exit
(1); }
   }
}
```

3. Comportmento da solução com um processo A e um processo B

ele funciona corretamente. Como A espera a resposta antes de sortear o próximo par, existe no máximo uma mensagem em cada fila e a soma recebida sempre corresponde ao pedido feito. Cerca de uma soma por segundo, por causa do sleep(1) de B. Saída de A em um teste:

```
A[302]: 40 + 17 = 57
A[302]: 80 + 66 = 146
A[302]: 34 + 73 = 107
A[302]: 83 + 71 = 154
```

4. Com dois processos A e um processo B

aparecem somas erradas. As duas instâncias de A leem da mesma fila de resposta, então um A pode receber a soma que B calculou para o outro.

Em um teste de 11 segundos, 2 de 10 respostas estavam trocadas:

```
A[322]: 80 + 36 = 137   <-- ERRADO (soma de outro pedido)
A[323]: 57 + 80 = 116   <-- ERRADO (soma de outro pedido)
```

Note que 80 + 36 = 116 e 57 + 80 = 137: as duas somas foram trocadas entre A[322] e A[323]. A causa é que a fila de resposta não identifica para quem é cada mensagem.

5. Com um processo A e dois processos B

Nenhuma soma errada foi observada (0 erros em 7 respostas). Cada pedido é consumido por apenas um dos B, e como só há um A, toda resposta volta para o dono certo. Também não houve ganho de desempenho: A é síncrono (envia e fica bloqueado esperando a resposta), então só existe um pedido em andamento por vez e o segundo B fica ocioso. O segundo B só ajudaria se houvesse mais de um A.

6. Dois processos A e dois processos B

O problema do item 4 reaparece: em um teste de 11 segundos, 4 de 20 respostas estavam erradas. O segundo B não corrige nada, porque o defeito está no lado das respostas (todos os A disputam a mesma fila). Exemplo:

```
A[366]: 57 + 80 = 116   <-- ERRADO (soma de outro pedido)
A[366]: 19 + 11 = 116   <-- ERRADO (soma de outro pedido)
A[367]: 80 + 36 = 137   <-- ERRADO (soma de outro pedido)
A[367]: 99 + 17 = 30    <-- ERRADO (soma de outro pedido)
```

7. Como resolver os problemas da forma mais adequada:

O problema ocorre sempre que há mais de um A, e vem de várias instâncias consumirem da mesma fila de resposta. A solução mais adequada é dar a cada processo A a sua própria fila de resposta, e fazer o pedido informar para qual fila B deve responder:

- Cada A cria uma fila de resposta com nome único, usando o seu PID (por exemplo /fila-mirelle-resposta-1234).
- A mensagem de pedido passa a ter três campos: pid do A, n1 e n2 (pedido_id_t).
- B recebe o pedido, calcula a soma, abre a fila de resposta do PID recebido (mq_open sem O_CREAT), envia a soma e fecha a fila.
- Ao terminar, A remove a sua fila com mq_unlink (tratando SIGINT/SIGTERM).

Com isso, a fila de pedidos continua única e compartilhada (vários B podem competir por ela sem problema, pois cada pedido é entregue a um só B), e cada resposta chega ao A que a pediu.

A alternativa de usar prioridades de mensagem para distinguir os destinatários não é adequada, pois mq_receive sempre devolve a mensagem de maior prioridade, e não a de uma prioridade escolhida. Testando a versão corrigida com 2 A e 2 B por 11 segundos, 0 de 20 respostas ficaram erradas:

```
A[388]: 25 + 97 = 122
A[388]: 45 + 44 = 89
A[389]: 40 + 32 = 72
A[389]: 81 + 5 = 86
B[384]: recebeu 25 e 97 de A[388]
B[384]: recebeu 78 e 96 de A[389]
```

processo_a_corrigido.c (processo A, versão corrigida)

```
// Processo A (versao corrigida: uma fila de resposta por processo A)
#include "filas.h"

static char nome_resp[64];

static void termina (int s) { mq_unlink (nome_resp); _exit (0); }

int main (void)
{
   srandom (time (NULL) ^ getpid ());
   snprintf (nome_resp, sizeof (nome_resp), "/fila-mirelle-resposta-%d", getpid ());
   signal (SIGINT, termina);
   signal (SIGTERM, termina);

   mqd_t pedido   = abre_fila (FILA_PEDIDO, O_WRONLY, sizeof (pedido_id_t));
   mqd_t resposta = abre_fila (nome_resp,   O_RDONLY, sizeof (int));
```

```
   while (1)
   {
      pedido_id_t p = { getpid (), random () % 100, random () % 100 };
      int soma;
      if (mq_send (pedido, (char *) &p, sizeof (p), 0) < 0) { perror ("mq_send"); exit (1); }
      if (mq_receive (resposta, (char *) &soma, sizeof (soma), NULL) < 0) { perror
("mq_receive"); exit (1); }
      printf ("A[%d]: %d + %d = %d%s\n", getpid (), p.n1, p.n2, soma,
              (p.n1 + p.n2 == soma) ? "" : "   <-- ERRADO");
      fflush (stdout);
   }
}
```

processo_b_corrigido.c (processo B, versão corrigida)

```
// Processo B (versao corrigida: responde na fila do A que fez o pedido)
#include "filas.h"

int main (void)
{
   mqd_t pedido = abre_fila (FILA_PEDIDO, O_RDONLY, sizeof (pedido_id_t));

   while (1)
   {
      pedido_id_t p;
      if (mq_receive (pedido, (char *) &p, sizeof (p), NULL) < 0) { perror ("mq_receive");
exit (1); }
      int soma = p.n1 + p.n2;
      printf ("B[%d]: recebeu %d e %d de A[%d]\n", getpid (), p.n1, p.n2, p.pid);
      fflush (stdout);
      sleep (1);

      char nome[64];
      snprintf (nome, sizeof (nome), "/fila-mirelle-resposta-%d", p.pid);
      mqd_t resposta = mq_open (nome, O_WRONLY);
      if (resposta == (mqd_t) -1) { perror ("mq_open (resposta)"); continue; }  // A ja
terminou
      if (mq_send (resposta, (char *) &soma, sizeof (soma), 0) < 0) perror ("mq_send");
      mq_close (resposta);
   }
}
```

Observações de uso

- Compilação: gcc -Wall processo_a.c -o processo_a -lrt (idem para processo_b, processo_a_corrigido e processo_b_corrigido).
- As filas ficam visíveis em /dev/mqueue (ll /dev/mqueue). Para apagar as filas dos testes: rm /dev/mqueue/fila-mirelle-* (o script limpa.sh faz isso).
- Teste com 2A+2B: ./processo_b_corrigido & ./processo_b_corrigido & ./processo_a_corrigido & ./processo_a_corrigido &
- mq_open de filas POSIX exige Linux (ou WSL no Windows). No Windows puro não existe mqueue.h.

Atividade 6 - Coordenação por espera ocupada

A implementação:

Um único programa (espera_ocupada.c) com duas threads POSIX, que usa a fila circular da Atividade 1 (queue.c/queue.h, lista circular duplamente encadeada). A fila tem 10 elementos, inicialmente cheia de valores aleatórios. Cada thread retira o primeiro elemento, sorteia um novo valor, coloca no fim da fila e imprime a operação no formato pedido. O modo é escolhido na linha de comando:

- ./espera_ocupada 0 <segundos> <imprime>: sem sincronização
- ./espera_ocupada 1 <segundos> <imprime>: alternância estrita
- ./espera_ocupada 2 <segundos> <imprime>: algoritmo de Peterson

O terceiro argumento (1 ou 0) liga ou desliga a impressão, o que é usado nas medições. O elemento retirado é reaproveitado (nó removido, valor trocado e reinserido no fim), então não há malloc na seção crítica. Compilação: gcc -O2 -Wall -pthread espera_ocupada.c queue.c -o espera_ocupada.

As variáveis de coordenação (vez, quer[], turno) usam C11 atomics com consistência sequencial. Isso é necessário no Peterson: com variáveis comuns (mesmo volatile), o compilador e o processador podem reordenar a escrita em quer[t] e a leitura de quer[o], e a exclusão mútua deixa de valer em processadores modernos. Para a execução poder terminar, o laço de espera também verifica a flag "parar", ligada pela main após o tempo definido.

espera_ocupada.c

```
// Atividade 6 - Coordenacao por espera ocupada
// Uso: ./espera_ocupada <modo> <segundos> <imprime>
//   modo: 0 = sem sincronizacao, 1 = alternancia estrita, 2 = Peterson
//   segundos: duracao da execucao (padrao 5)
//   imprime: 1 = imprime cada operacao (padrao), 0 = modo silencioso
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdatomic.h>
#include <time.h>
#include <unistd.h>
#include "queue.h"

#define TAM 10

typedef struct item_t
{
   struct item_t *prev, *next;   // deve ser o 1o campo (compativel com queue_t)
   int valor;
} item_t;

static item_t itens[TAM];
static item_t *fila = NULL;

static int modo = 0, imprime = 1;
static atomic_int parar = 0;

// alternancia estrita
static atomic_int vez = 0;
// Peterson
static atomic_int quer[2] = {0, 0};
static atomic_int turno = 0;

static long insercoes[2][16];    // contadores (separados para evitar false sharing)
static long erros[2][16];        // falhas detectadas pela biblioteca de fila



static void entra (int t)
{
   int o = 1 - t;
```

```
   if (modo == 1)
   {
      while (atomic_load (&vez) != t && !atomic_load (&parar)) ;
   }
   else if (modo == 2)
   {
      atomic_store (&quer[t], 1);
      atomic_store (&turno, o);
      while (atomic_load (&quer[o]) && atomic_load (&turno) == o && !atomic_load (&parar)) ;
   }
}

static void sai (int t)
{
   if (modo == 1) atomic_store (&vez, 1 - t);
   else if (modo == 2) atomic_store (&quer[t], 0);
}

static void *tarefa (void *arg)
{
   int t = (int) (long) arg;
   unsigned int semente = 1234 + t;

   while (!atomic_load (&parar))
   {
      entra (t);
      if (modo != 0 && atomic_load (&parar)) { sai (t); break; }

      item_t *it = fila;                  // primeiro elemento
      int velho = it ? it->valor : -1;
      if (queue_remove ((queue_t **) &fila, (queue_t *) it) < 0) { erros[t][0]++; sai (t);
continue; }
      int novo = rand_r (&semente) % 100;
      it->valor = novo;
      if (queue_append ((queue_t **) &fila, (queue_t *) it) < 0) { erros[t][0]++; sai (t);
continue; }
      insercoes[t][0]++;

      if (imprime)
      {
         printf ("thread %d: tira %d, p\303\265e %d, fila: ", t, velho, novo);
         if (fila)
         {
            item_t *p = fila; int n = 0;
            do { printf ("%d ", p->valor); p = p->next; } while (p && p != fila && ++n < 64);
         }
         printf ("\n");
      }
      sai (t);
   }
   return NULL;
}

int main (int argc, char **argv)
{
   int segundos = 5;
   if (argc > 1) modo = atoi (argv[1]);
   if (argc > 2) segundos = atoi (argv[2]);
   if (argc > 3) imprime = atoi (argv[3]);

   srand (42);
   for (int i = 0; i < TAM; i++)
   {
      itens[i].valor = rand () % 100;
      itens[i].prev = itens[i].next = NULL;
      queue_append ((queue_t **) &fila, (queue_t *) &itens[i]);
   }

   pthread_t th[2];
   struct timespec a, b;
   clock_gettime (CLOCK_MONOTONIC, &a);
   for (long i = 0; i < 2; i++) pthread_create (&th[i], NULL, tarefa, (void *) i);
   sleep (segundos);
```

```
   atomic_store (&parar, 1);
   for (int i = 0; i < 2; i++) pthread_join (th[i], NULL);
   clock_gettime (CLOCK_MONOTONIC, &b);

   double dt = (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / 1e9;
   long total = insercoes[0][0] + insercoes[1][0];
   int tam = queue_size ((queue_t *) fila);

   fprintf (stderr, "\n--- resultado (modo %d) ---\n", modo);
   fprintf (stderr, "tempo: %.3f s\n", dt);
   fprintf (stderr, "insercoes: thread 0 = %ld, thread 1 = %ld, total = %ld\n",
            insercoes[0][0], insercoes[1][0], total);
   fprintf (stderr, "insercoes por segundo: %.0f\n", total / dt);
   fprintf (stderr, "erros detectados na fila: %ld\n", erros[0][0] + erros[1][0]);
   fprintf (stderr, "tamanho final da fila: %d (esperado %d)\n", tam, TAM);
   return 0;
}
```

O queue.c usado segue a API do PingPongOS (queue_append, queue_remove, queue_size, queue_print), com verificações extras que impedem laços infinitos caso a fila fique corrompida; elas apenas registram um erro. Pode ser trocado pelo queue.c da própria Atividade 1.

1. Sem mecanismo de sincronização:

Sem coordenação, as operações retirar e inserir não são atômicas: cada uma altera vários ponteiros (prev e next de nós vizinhos e o ponteiro de início da fila). Se uma thread for interrompida, ou se as duas executarem ao mesmo tempo em núcleos diferentes, no meio dessas operações, a fila pode ficar inconsistente: a mesma posição pode ser retirada pelas duas threads, elementos podem ser perdidos ou duplicados, o tamanho deixa de ser 10 e a lista pode deixar de ser circular. É uma condição de corrida.

Ressalva sobre o meu teste: o ambiente onde rodei o programa tinha apenas 1 vCPU, então as duas threads nunca executam de fato em paralelo, e a troca de thread só acontece no fim da fatia de tempo, o que raramente cai no meio de uma operação. Por isso nessa máquina a fila se comportou corretamente (0 erros, tamanho final 10) mesmo sem sincronização. Isso não prova que o código esteja correto: a corrida é real e deve aparecer em máquinas com vários núcleos (como o notebook), onde o programa deve ser executado para observar o problema. A saída (sem sincronização, com a ordem das threads qualquer) começa assim:

```
thread 0: tira 66, põe 7, fila: 40 81 41 12 58 21 40 35 43 7
thread 0: tira 40, põe 7, fila: 81 41 12 58 21 40 35 43 7 7
thread 0: tira 81, põe 3, fila: 41 12 58 21 40 35 43 7 7 3
thread 0: tira 41, põe 59, fila: 12 58 21 40 35 43 7 7 3 59
```

2. Alternância estrita

Uma variável compartilhada "vez" indica qual thread pode entrar na seção crítica. A thread t espera (espera ocupada) até vez == t, executa a operação e passa a vez para a outra. A fila se comportou corretamente: o tamanho final foi 10, sem erros, e a ordem de execução é exatamente t0 t1 t0 t1:

```
thread 0: tira 66, põe 7, fila: 40 81 41 12 58 21 40 35 43 7
thread 1: tira 40, põe 37, fila: 81 41 12 58 21 40 35 43 7 37
thread 0: tira 81, põe 7, fila: 41 12 58 21 40 35 43 7 37 7
thread 1: tira 41, põe 83, fila: 12 58 21 40 35 43 7 37 7 83
```

A desvantagem é que as threads são obrigadas a se alternar: uma thread não pode entrar duas vezes seguidas mesmo que a outra não queira entrar, e a thread mais rápida fica sempre esperando a mais lenta.

3. Algoritmo de Peterson

Cada thread sinaliza que quer entrar (quer[t] = 1), cede o turno à outra (turno = o) e espera enquanto a outra quiser entrar e o turno for dela. Ao sair, faz quer[t] = 0. A fila se comportou corretamente (tamanho final 10, 0 erros). Diferente da alternância, o Peterson não obriga a ordem

t0 t1: se a outra thread não está interessada, a mesma thread pode entrar várias vezes seguidas. No teste, a thread 0 fez 2441 inserções e a thread 1 fez 257, o que mostra essa flexibilidade (e também o efeito da fatia de tempo em uma única CPU).

4. Medição do número de inserções por segundo

Cada thread tem um contador de inserções bem-sucedidas (contadores em linhas de cache diferentes, para não atrapalhar a medição). O programa roda por um tempo fixo (argumento segundos), a main mede o tempo real decorrido com clock_gettime(CLOCK_MONOTONIC) e calcula inserções por segundo = (inserções da thread 0 + inserções da thread 1) / tempo decorrido. A medição é feita de duas formas: com a impressão desligada (mede o custo da fila e da sincronização) e com impressão ligada, redirecionada para /dev/null (inclui o custo do printf, mais parecido com o uso normal). O script medir.sh executa cada configuração 3 vezes por 3 segundos.

5. Desempenho das três implementações

Ambiente em que as medidas foram feitas: Linux (Ubuntu 24.04), gcc 13.3, -O2, Intel Xeon 2,8 GHz com 1 vCPU. Resultados (inserções por segundo, 3 repetições):

Versão	Rep. 1	Rep. 2	Rep. 3	Faixa observada
Sem sincronização (sem imprimir)	11,6 milhões	73,0 milhões	74,1 milhões	muito variável
Alternância estrita (sem imprimir)	251	251	250	≈ 250
Peterson (sem imprimir)	36.962	40.603	39.303	≈ 37 a 41 mil
Sem sincronização (imprimindo)	1.372.189	1.303.572	1.316.527	≈ 1,3 milhão
Alternância estrita (imprimindo)	252	252	250	≈ 250
Peterson (imprimindo)	1.581	2.198	1.614	≈ 1,6 a 2,2 mil

Justificativa dos resultados

- Sem sincronização é a mais rápida, pois não tem nenhum custo de coordenação: o laço só executa a operação da fila. Mas essa velocidade não vale nada em máquinas com vários núcleos, porque a fila pode ser corrompida. A variação grande entre as repetições (11,6 a 74 milhões) vem do escalonamento do sistema com 1 CPU.
- Alternância estrita deu cerca de 250 inserções por segundo nas duas configurações. Com uma única CPU, a thread que não tem a vez fica girando no laço de espera e só sai dele quando o escalonador a interrompe, ao fim da sua fatia de tempo (na ordem de 4 ms). Isso dá uma troca de thread por fatia, ou seja, cerca de 1000/4 = 250 inserções por segundo; o valor praticamente não muda com ou sem impressão, pois o gargalo é a espera ocupada, não o trabalho útil. É o pior caso da espera ocupada: a CPU é gasta girando, e não trabalhando. Em uma máquina com vários núcleos o valor seria bem maior, mas ainda limitado pela obrigação de alternar e pela troca da variável "vez" entre os caches dos núcleos.
- Peterson ficou entre os dois: cerca de 40 mil por segundo sem imprimir. Como não exige alternância, uma thread pode executar muitas operações seguidas quando a outra não está interessada (o que explica a divisão 2441 x 257 vista acima), então só paga o custo de espera quando há disputa de fato. É mais lento que a versão sem sincronização porque cada entrada e saída executa escritas atômicas com consistência sequencial (instruções mais caras, com barreira de memória) e porque, quando uma thread é interrompida dentro da seção crítica, a outra gasta sua fatia inteira girando.
- Com impressão ligada, o custo do printf (formatação e chamada de escrita) passa a dominar. A versão sem sincronização cai de dezenas de milhões para cerca de 1,3 milhão

por segundo; no Peterson, o printf é executado dentro da seção crítica, o que aumenta o tempo em que a outra thread fica esperando, e o valor cai para cerca de 2 mil por segundo.

Resumo da ordem de desempenho nesse ambiente: sem sincronização > Peterson > alternância estrita. Em termos de correção em máquinas multinúcleo: só as duas últimas garantem a exclusão mútua. Peterson é preferível à alternância estrita porque não impõe ordem e não deixa uma thread esperar sem necessidade.

Observação importante: como esse ambiente tinha só 1 CPU, os números absolutos (principalmente de alternância e Peterson) não representam o que acontece em um computador com vários núcleos. Os programas e o script medir.sh foram entregues para que as medidas sejam repetidas na sua máquina, e os valores da tabela podem ser substituídos pelos novos.

queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

// Fila circular duplamente encadeada (API no estilo PingPongOS)
typedef struct queue_t
{
   struct queue_t *prev;
   struct queue_t *next;
} queue_t;

int queue_size (queue_t *queue);
void queue_print (char *name, queue_t *queue, void print_elem (void *));
int queue_append (queue_t **queue, queue_t *elem);
int queue_remove (queue_t **queue, queue_t *elem);

#endif
```

medir.sh

```
#!/bin/bash
# Mede insercoes por segundo das tres versoes (3 repeticoes de 3 s cada)
# Uso: ./medir.sh
for imp in 0 1; do
  for m in 0 1 2; do
    for r in 1 2 3; do
      v=$(./espera_ocupada $m 3 $imp 2>&1 >/dev/null | grep "por segundo" | awk '{print $4}')
      echo "imprime=$imp modo=$m rep=$r ips=$v"
    done
  done
done
```

